RFC: Layered Parser Design - Fall Draft

Author: Leo Sallos

Date: 2026-10-23

Project: 009 - Layered Functional Programming Language

Status: Proposed

Problem

For the layers of our programming language to be implemented, we need a parser that can adapt for many layers of functionality. The team has already decided that constructing multiple languages, one for each layer, is not the way to go, so we need a modular solution to out parser. This must be made soon, as development of the language cannot begin without this problem getting solved. A requirement for this problem to be solved is the ability to use the Happy parser generator with our design, so that out design remains within the 30-week time frame.

Options

Option 1: Do Nothing

Do nothing about the issue. If this option is chosen, then we don't have a project that has the set out features.

Option 2: Ad-hoc Modularity with Monolithic Parser and Post-parse Blacklist

Build a parser that is feature complete, as in all parser features are implemented. After the interpreter parses, run the generated AST through a blacklist that throws error if a feature is disabled.

The benefits of this option are the simplicity to implement, increase in runtime complexity only scales with the size of the program. The costs of this option are that we have less control over the design and specification of the parser.

Option 3: Combinational Multi-Staged Parser

A parser that is broken into many stages (expressions, statements, declarations), and within each of those stages is a set of modules in order of priority that each contains a set of features for the parser and generates a sub-AST. Upon one module failing, it moves down the chain until one module returns successfully, or all modules fail, resulting in the error of the original module. For each set of features, certain modules will be activated and put in a predetermined set of priority.

The benefits of this option is that this provides a significant level of control over the parser for each layer of features. The costs is that this would be difficult to implement,

come with runtime complexity that scales with the number of parser features, and implementing error messages would be significantly more difficult in this method.

Option 4: IFDEF Parser Modularity

The parser would be written with `IFDEF` blocks that would separate different parser features depending on compiler flags. Our interpreter would come with all valid combinations of our language features precompiled, and switch between which parser is used depending on the user runtime specification.

The benefits of this option is that this is easier to implement than the combinational parser and with the same level of control over the layer features. This option would also be more performant at runtime than the alternative. The costs of this option is that this leads to longer compile time for binary size for the interpreter, and this increases code complexity significantly as more features are added to the parser.

Analysis

Option 1 is out of the question, as our language would not have the layered features, which is the whole point of the project.

Complexity to implement the parser is a significant issue, as our team does not have the time or resources to reinvent the wheel in 30 weeks. So comparing the two easiest to implement options of 2 & 4, 4 seems to be the superior option. The benefits that option 2 has over 4 is that the implementation will result in less code complexity as the number of features increase. For what they have on equal footing is the implementation of error messages, and the high-level design are on about equal footing. The most important factors are what option 4 does better, and it is that 4 offers more control over the parser design in being about to have some features overwrite others with relative ease. That fact is what puts option 4 above 2, so option 2 should be out of consideration.

Comparing options 3 and 4, option 4 has the immediate downsides of increased code complexity and the need to compile many iterations of the parser depending on all possible feature configurations. But the main benefit of two is that it is the most straight-forward option to implement, meaning that development will move forward very predictably. The biggest unknown between these option is which will be better for developing a good error system. We as a team have not yet decided on an error system, so how we would develop such a system is still up in the air, so knowing the best of these two options is not yet possible. The biggest benefit of option 3, is that you get the same level of expressiveness in the parser design, but without the multiple compilations.

Recommendation & Verification Plan

I recommend option 3. It may be the most experimental, but it is tied with the most parser expressiveness, and has the most straight-forward compilation and deployment for the language.

A verification plan to see if it is working would be to develop a plan for the error system, and implement it in the base (least featureful) version of our language. If the team can implement an error system that meets out project criteria and not run into issues with the parser architecture, then the choice was successful. Otherwise, a switch to option 4 may be needed, as we can keep a similar plan for the modularity of the features, as it is just as expressive, but with a different implementation, so it would be the least costly fallback.

Constraints

Out only constraint is the time to implement the feature to get a sense as to if it was the right decision.

Delegation & Validations

CI tools/test suite will be needed to make sure that the designed parser: produces a valid parse output (works as intended), and that it implements the error system correctly as development continues.

AI tools were not used in the making of the document, or in the brainstorming, analysis, and decisions presented in this document.